

Memory Retrieval Scoring with Recency Decay, Worked Out by Hand

Why a slightly-less-similar *fresh* memory should beat a highly-similar *stale* one — the recall ranker of Module 4.4, every number counted by hand.

What this document does. It takes the recall step from Module 4.4 — the `recall()` query that ranks a user’s memories before injecting them into the agent prompt — and works it on *four* candidate memories for the Customer-Support Resolver: a gold-tier refund rule, an email-contact preference, a recent INV-1102 question, and a stale old-address fact. We fix each memory’s cosine similarity and age, pick the blend weights, derive the recency-decay constant from a half-life, and then compute the combined score for every memory by hand. The module ranks memory by $0.6 \cdot \text{sim} + 0.2 \cdot \text{salience} + 0.2 \cdot \text{recency}$ with a 7-day decay; we rebuild that idea from first principles. Every number is reproduced by the small Python program in Appendix A.

Where this sits. This is **Topic 1 of Module 4.4** (Long-Term Memory Architectures) — the math behind stage 05, “Rank: semantic + recency + salience, top-*k* under budget.” Its companions: Module 4.4 Topic 2 (salience decay and TTL eviction — the *same* exponential, used to forget instead of to rank) and Module 4.3 (context budgeting — why recall is capped at top-*k* in the first place).

Concepts covered, in order: cosine similarity as a raw score · the exponential recency kernel $r(\Delta) = e^{-\lambda \Delta}$ · the half-life relation $\lambda = \ln 2 / t_{1/2}$ · the convex blend $s = \alpha \text{sim} + \beta r$ · ranking four memories by hand · a stale high-similarity memory losing to a fresh one · how the ranking flips as the half-life changes · recency as the dial that keeps memory current.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: four memories, small enough to rank by hand

When the Resolver starts a turn, the memory store may hold dozens of facts and events about this user. It cannot inject them all — the recall block has a token budget — so it must *score* each candidate and keep the top few. The naive score is pure semantic similarity: how close is this memory’s embedding to the current query? That surfaces the *most relevant* memory, but not necessarily the most *useful*: a year-old fact can be a perfect semantic match and still be wrong today (the user moved cities). Module 4.4 fixes this by blending similarity with a *recency* term that decays with age.

We take four candidate memories for one user. Each has a cosine similarity to the current query (a number in $[0, 1]$, already computed by the vector store) and an age in days. These are illustrative, chosen round so the sums follow on paper; the mechanics are identical at any scale.

Tag	Memory	Type	sim	age (days)
M1	gold-tier refund rule	semantic	0.91	30
M2	prefers email contact	semantic	0.86	2
M3	asked about INV-1102	episodic	0.78	1
M4	old address: Osaka	semantic	0.94	14

The weights. We blend with $\alpha = 0.7$ on similarity and $\beta = 0.3$ on recency (the module uses 0.6/0.2/0.2 across three terms; we fold salience aside and keep two so the arithmetic is hand-followable). Recency uses a half-life of $t_{1/2} = 7$ days — a memory loses half its freshness weight every week.

Notation. $\text{sim}_i \in [0, 1]$ is memory i 's cosine similarity; Δ_i is its age in days; $r_i = r(\Delta_i)$ its recency score; $s_i = \alpha \text{sim}_i + \beta r_i$ its combined score. We carry four decimals throughout.

Intuition

Similarity asks “does this memory *match* the question?” Recency asks “is this memory still *true*?” A customer's old shipping address matches a shipping question perfectly — high similarity — but if they moved last week it is exactly the memory you must *not* surface. Pure similarity has no clock. Recency decay gives the ranker one: the older a memory, the more its match has to overcome to stay on top.

2 Step 1 — the recency kernel and its half-life

We want a freshness score that is 1 for a brand-new memory and falls smoothly toward 0 as the memory ages, never going negative. The exponential does exactly this:

$$r(\Delta) = e^{-\lambda \Delta}, \quad \lambda > 0, \Delta \geq 0. \quad (1)$$

At $\Delta = 0$, $r = 1$ (today's memory is fully fresh). As $\Delta \rightarrow \infty$, $r \rightarrow 0$. The single knob λ sets *how fast* freshness drains. Rather than pick λ directly, we pick the more intuitive **half-life** $t_{1/2}$: the age at which recency has fallen to one half. Set $r(t_{1/2}) = \frac{1}{2}$ and solve:

$$e^{-\lambda t_{1/2}} = \frac{1}{2} \implies -\lambda t_{1/2} = \ln \frac{1}{2} = -\ln 2 \implies \lambda = \frac{\ln 2}{t_{1/2}} \quad (2)$$

With $t_{1/2} = 7$ days, $\lambda = \ln 2 / 7 = 0.6931 / 7 = 0.0990$ per day.

Mini-example — this operation in isolation

Check the half-life directly. $r(7) = e^{-0.0990 \cdot 7} = e^{-0.6931} = 0.5000$. ✓ Exactly one half, as designed. And because the exponential halves on every additional $t_{1/2}$, the values come out round: $r(14) = e^{-2 \ln 2} = \frac{1}{4} = 0.2500$, $r(21) = \frac{1}{8}$, and so on. A memory two weeks old keeps a quarter of its freshness weight; one three weeks old, an eighth. The kernel is just repeated halving.

3 Step 2 — recency for each memory

Apply Equation (1) with $\lambda = 0.0990$ to the four ages:

$$\begin{aligned} r_1 &= e^{-0.0990 \cdot 30} = e^{-2.9706} = 0.0513, & r_2 &= e^{-0.0990 \cdot 2} = e^{-0.1980} = 0.8203, \\ r_3 &= e^{-0.0990 \cdot 1} = e^{-0.0990} = 0.9057, & r_4 &= e^{-0.0990 \cdot 14} = e^{-1.3863} = 0.2500. \end{aligned}$$

The spread is already telling. M3 (one day old) keeps 91% of its freshness; M1 (thirty days old) keeps barely 5%. M4, at exactly two half-lives, sits at a clean 0.25. Similarity ordered these

as $M4 > M1 > M2 > M3$; recency reverses that almost completely, because the high-similarity memories here are the *old* ones.

4 Step 3 — the combined score

Now blend. The combined score is the convex combination

$$s_i = \alpha \text{sim}_i + \beta r_i, \quad \alpha + \beta = 1, \quad \alpha = 0.7, \quad \beta = 0.3. \quad (3)$$

Because $\alpha + \beta = 1$ and both inputs lie in $[0, 1]$, every score also lies in $[0, 1]$ — the blend never inflates a memory past what its parts allow. Working all four:

Tag	Memory	sim	age	r_i	αsim_i	βr_i	s_i
M2	prefers email	0.86	2	0.8203	0.6020	0.2461	0.8481
M3	asked INV-1102	0.78	1	0.9057	0.5460	0.2717	0.8177
M4	old address	0.94	14	0.2500	0.6580	0.0750	0.7330
M1	gold-tier rule	0.91	30	0.0513	0.6370	0.0154	0.6524

Sample check, M2: $\alpha \text{sim} = 0.7 \times 0.86 = 0.6020$; $\beta r = 0.3 \times 0.8203 = 0.2461$; sum = 0.8481. ✓

The rows are printed in *final* rank order. Read the two orderings side by side:

Rank	Pure similarity	Similarity + recency
1	M4 (sim 0.94)	M2 ($s = 0.8481$)
2	M1 (sim 0.91)	M3 ($s = 0.8177$)
3	M2 (sim 0.86)	M4 ($s = 0.7330$)
4	M3 (sim 0.78)	M1 ($s = 0.6524$)

This is the whole point of the document. Under pure similarity, **M4 — the stale “old address: Osaka” fact — ranks first.** It is the best semantic match, so a similarity-only ranker would inject an address the user abandoned two weeks ago. Once recency is folded in, M4 falls to *third*, and **M2, a slightly-less-similar but fresh preference, wins.** The fresh, slightly-weaker match beat the stale, stronger one — exactly the behaviour recency weighting exists to produce.

Watch out

Recency is a tie-breaker, not a veto. M4 dropped two places but did *not* vanish — its high similarity still kept it above the thirty-day-old M1. If you crank β too high (or the half-life too short), you flip the failure mode: the ranker becomes recency-obsessed and surfaces whatever is newest even when it barely matches the query. The blend $\alpha = 0.7$, $\beta = 0.3$ keeps similarity in charge and lets recency *break ties among comparably relevant* memories. Tune both knobs against real recall traffic, not by feel.

5 Step 4 — how the ranking flips with the half-life

The half-life is the dial. A *short* half-life punishes age hard, so freshness dominates; a *long* half-life lets old-but-relevant memories survive. Hold M2 (fresh, sim = 0.86, age 2) against M4 (stale,

sim = 0.94, age 14) and sweep $t_{1/2}$, recomputing $\lambda = \ln 2/t_{1/2}$ each time:

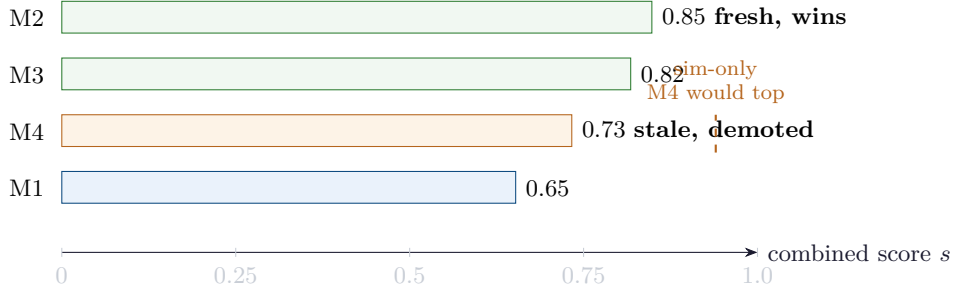
$t_{1/2}$	λ	$r(\text{M2, age 2})$	$r(\text{M4, age 14})$	$s_{\text{M2}} / s_{\text{M4}}$	top
3	0.231	0.6300	0.0394	0.7910 / 0.6698	M2 (fresh)
7	0.099	0.8203	0.2500	0.8481 / 0.7330	M2 (fresh)
14	0.0495	0.9057	0.5000	0.8737 / 0.8080	M2 (fresh)
30	0.0231	0.9548	0.7236	0.8885 / 0.8751	M2 (fresh)
60	0.0116	0.9772	0.8507	0.8951 / 0.9132	M4 (stale)

Read it top to bottom. For any reasonable half-life — days to a month — the fresh memory M2 stays on top: its recency edge more than pays for its 0.08 similarity deficit. Only when the half-life stretches to *sixty days* — so long that a two-week-old memory is still 85% fresh — does recency stop mattering enough, and M4’s raw similarity finally wins. Sample check, $t_{1/2} = 60$: $\lambda = \ln 2/60 = 0.01155$; $s_{\text{M4}} = 0.7 \cdot 0.94 + 0.3 \cdot e^{-0.01155 \cdot 14} = 0.658 + 0.3 \cdot 0.8507 = 0.9132$. ✓

Intuition

The half-life encodes *how fast your domain goes stale*. A breaking-news assistant wants a half-life of hours: yesterday’s fact is already suspect. A user-preference store wants weeks or months: “prefers email” does not rot overnight. The same exponential kernel serves both — you only move $t_{1/2}$. The flip table above is that intuition made arithmetic: short half-life $\Rightarrow$ fresh wins; long half-life $\Rightarrow$ the ranker behaves like pure similarity again.

6 The ranking, drawn



Green bars are fresh memories, the orange bar is the stale high-similarity memory M4. The dashed orange tick marks where M4 *would* sit on a pure-similarity axis — first place, furthest right. Folding in recency shrinks M4’s bar back to third. The picture *is* Equation (3): each bar is $\alpha \text{sim} + \beta r$, and recency is what pulls the stale memory leftward.

7 Recall-scoring cheat-sheet

Recency kernel	$r(\Delta) = e^{-\lambda\Delta}, \quad r(0) = 1, r(\infty) = 0$
Half-life $\rightarrow$ rate	$\lambda = \ln 2 / t_{1/2}$
Half-life check	$r(t_{1/2}) = \frac{1}{2}; \quad r(2t_{1/2}) = \frac{1}{4}$
Combined score	$s = \alpha \text{ sim} + \beta r, \quad \alpha + \beta = 1$
Bounded output	$\text{sim}, r \in [0, 1] \Rightarrow s \in [0, 1]$
Three-term form (module)	$s = 0.6 \text{ sim} + 0.2 \text{ salience} + 0.2 r$
Short $t_{1/2}$	freshness dominates — newest wins
Long $t_{1/2}$	decays slowly — ranks like pure similarity

8 An honest word about these numbers

The four similarities, the four ages, and the weights ($\alpha = 0.7$, $\beta = 0.3$, $t_{1/2} = 7$ days) are illustrative round numbers, chosen so the recency values land on clean fractions ($\frac{1}{2}$, $\frac{1}{4}$) and the ranking flip is easy to see by hand. Real cosine similarities come out of your embedding model and rarely sit on tidy values; real ages are continuous; and a production ranker carries the third *salience* term the module shows, plus a hard token budget on top. What is *not* illustrative is the structure: recency is an exponential with a half-life you choose, the blend is a convex combination that stays bounded, and a sufficiently stale memory loses to a fresher one even at lower similarity. Move the constants and the exact scores shift; the shape — fresh-beats-stale, tunable by one half-life dial — does not. That shape is why every serious memory store ranks on more than similarity alone.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the recency values, the per-memory scores, both rankings, and the half-life flip table; change the similarities, ages, weights, or `t_half` to explore your own ranker.

```

1 # recall_scoring.py -- reproduces every number in "Memory Retrieval Scoring".
2 import math
3
4 ALPHA, BETA = 0.7, 0.3 # blend weights on similarity / recency (sum to 1)
5 T_HALF = 7.0 # recency half-life, days
6 LAM = math.log(2) / T_HALF # decay rate = ln2 / t_half
7
8 def recency(age_days): # r(d) = exp(-lambda * d)
9     return math.exp(-LAM * age_days)
10
11 def score(sim, age_days): # s = a*sim + b*recency
12     return ALPHA * sim + BETA * recency(age_days)
13
14 # (tag, name, cosine similarity, age in days)
15 MEMS = [
16     ("M1", "gold-tier refund rule", 0.91, 30),
17     ("M2", "prefers email contact", 0.86, 2),
18     ("M3", "asked about INV-1102", 0.78, 1),
19     ("M4", "old address: Osaka", 0.94, 14),

```

```

20 ]
21
22 print(f"lambda = ln2/{T_HALF:.0f} = {LAM:.4f}")
23 print(f"half-life check: r({T_HALF:.0f}) = {recency(T_HALF):.4f} (should be 0.5)\n")
24
25 scored = []
26 for tag, name, sim, age in MEMS:
27     r = recency(age)
28     s = score(sim, age)
29     scored.append((tag, sim, age, r, s))
30     print(f"{tag} sim={sim:.2f} age={age:2d} r={r:.4f}"
31           f" a*sim={ALPHA*sim:.4f} b*r={BETA*r:.4f} s={s:.4f}")
32
33 print("\nPure-similarity ranking:")
34 for i, (tag, sim, *_ ) in enumerate(sorted(scored, key=lambda x: -x[1]), 1):
35     print(f" {i}. {tag} sim={sim:.2f}")
36
37 print("\nSimilarity + recency ranking:")
38 for i, row in enumerate(sorted(scored, key=lambda x: -x[4]), 1):
39     print(f" {i}. {row[0]} s={row[4]:.4f}")
40
41 print("\nFlip table: M2(fresh .86/2d) vs M4(stale .94/14d)")
42 for th in (3, 7, 14, 30, 60):
43     lam = math.log(2) / th
44     s2 = ALPHA*0.86 + BETA*math.exp(-lam*2)
45     s4 = ALPHA*0.94 + BETA*math.exp(-lam*14)
46     top = "M2(fresh)" if s2 > s4 else "M4(stale)"
47     print(f" t_half={th:2d} s_M2={s2:.4f} s_M4={s4:.4f} top={top}")
48
49 # Expected output:
50 # lambda = ln2/7 = 0.0990
51 # half-life check: r(7) = 0.5000 (should be 0.5)
52 #
53 # M1 sim=0.91 age=30 r=0.0513 a*sim=0.6370 b*r=0.0154 s=0.6524
54 # M2 sim=0.86 age= 2 r=0.8203 a*sim=0.6020 b*r=0.2461 s=0.8481
55 # M3 sim=0.78 age= 1 r=0.9057 a*sim=0.5460 b*r=0.2717 s=0.8177
56 # M4 sim=0.94 age=14 r=0.2500 a*sim=0.6580 b*r=0.0750 s=0.7330
57 #
58 # Pure-similarity ranking:
59 # 1. M4 sim=0.94
60 # 2. M1 sim=0.91
61 # 3. M2 sim=0.86
62 # 4. M3 sim=0.78
63 #
64 # Similarity + recency ranking:
65 # 1. M2 s=0.8481
66 # 2. M3 s=0.8177
67 # 3. M4 s=0.7330
68 # 4. M1 s=0.6524
69 #
70 # Flip table: M2(fresh .86/2d) vs M4(stale .94/14d)
71 # t_half= 3 s_M2=0.7910 s_M4=0.6698 top=M2(fresh)
72 # t_half= 7 s_M2=0.8481 s_M4=0.7330 top=M2(fresh)
73 # t_half=14 s_M2=0.8737 s_M4=0.8080 top=M2(fresh)
74 # t_half=30 s_M2=0.8885 s_M4=0.8751 top=M2(fresh)
75 # t_half=60 s_M2=0.8951 s_M4=0.9132 top=M4(stale)

```

— end of document —